

AI 综合开发岗位面试准备

适用岗位：AI 综合开发 / AI 应用开发 / 智能体搭建 / 全栈协作方向

参考项目：MostarManus AI Agent

准备重点：前后端基础、Spring Boot 接口、RAG、Function Calling、 workflows 搭建、接口联调、测试验证

1. 岗位理解

这个岗位不是纯算法岗，也不是纯前端或纯后端岗，而是偏 **AI 应用落地型开发**。

岗位要求可以拆成几类：

1. 前端基础

- 熟悉 HTML、CSS、JavaScript。
- 了解 Vue 基础使用。
- 能完成简单页面和交互开发。

2. 后端基础

- 掌握计算机网络、数据库、接口通信等基础。
- 了解 Spring Boot。
- 能进行基础接口开发、接口联调和数据库操作。

3. AI 应用能力

- 熟练使用 AI 工具。
- 能在扣子、Coze 等平台搭建智能体和自动化工作流。
- 能根据业务场景落地可用方案。
- 重点负责测试、跑通流程、验证功能可用性和稳定性。

4. 软素质

- 学习能力强。
- 沟通协作好。
- 主动性强。
- 能适应全栈式项目节奏。

面试表达重点：

我不是只会调用大模型 API，而是理解 AI 应用落地需要把业务流程、知识库、工具调用、接口联调和测试验证结合起来。

2. 自我介绍模板

面试官您好，我目前主要学习和实践方向是 Java 后端和 AI 应用开发。技术上我熟悉 Spring Boot、REST API、数据库基础、接口联调，也了解 HTML、CSS、JavaScript 和 Vue 的基础使用。

最近我做了一个 AI Agent 项目，叫 **MostarManus**，是基于 Spring Boot 3、Spring AI 和大模型接口实现的智能学习与任务协作助手。项目里实现了普通对话、SSE 流式响应、本地会话记忆、长期记忆、RAG 知识库检索、pgvector 向量存储、工具调用和安全拦截等功能。

我对 AI 应用落地比较感兴趣，也愿意使用扣子、Coze、工作流平台和大模型 API，把业务需求拆成智能体流程，再通过接口调试、效果测试和稳定性验证完成落地。

3. 高频面试问题与答案

3.1 你理解的 AI 应用开发是什么？

AI 应用开发不是单纯调用大模型接口，而是把业务流程、大模型能力、知识库、工具调用和用户交互结合起来。

比如一个智能客服或学习助手，不能只把用户问题发给模型，还需要根据业务知识库检索资料，通过 Prompt 约束回答范围，必要时调用工具完成查询、计算、保存任务等操作，最后还要测试回答准确性、稳定性和异常情况。

所以 AI 应用开发更像是：

业务需求分析 + Prompt 设计 + 工作流搭建 + 接口集成 + 测试验证。

3.2 你了解 RAG 吗？

了解。RAG 是 Retrieval-Augmented Generation，中文叫检索增强生成。

它的核心流程是：

1. 先把业务文档切分成多个片段。
2. 对片段做 embedding 向量化。
3. 存入向量数据库，比如 pgvector。
4. 用户提问时，也把问题向量化。
5. 从向量库中找出最相关的文档片段。
6. 把这些片段和用户问题一起交给大模型回答。

它的好处是可以减少模型幻觉，让回答基于企业自己的知识库。

我的 MostarManus 项目里就做了本地 Markdown 文档的读取、切分、入库和相似度检索，然后在对话前把相关知识拼进 Prompt。

3.3 Embedding 是什么？

Embedding 是把文本、图片等内容转换成向量，也就是一组数字。

向量可以表示语义特征。两个文本意思越接近，它们的向量距离通常越近。

在 RAG 里，文档片段和用户问题都会被转换成向量，然后通过相似度搜索找到最相关的内容。

3.4 向量数据库是什么？

向量数据库是用来存储和检索向量数据的数据库。

普通数据库更擅长精确查询，比如按照 ID 查用户；向量数据库更擅长语义相似度查询，比如用户问“怎么重置密码”，它可以找到文档中“忘记密码处理流程”相关内容。

我的项目里使用的是 PostgreSQL + pgvector，它可以在 PostgreSQL 中保存向量，并支持相似度检索。

3.5 Function Calling 是什么？

Function Calling 是让大模型在需要时调用外部工具或业务函数。

比如用户问“帮我计算 $128 * 36$ ”，模型不一定自己算，而是调用计算器工具。用户说“帮我保存一个学习计划”，模型可以调用任务保存工具。

它的价值是让 AI 从“只会回答”变成“可以执行操作”。

我的项目里定义过计算器、时间、天气、任务、学习计划、记忆等工具类，并通过 Spring AI 的 `@Tool` 注解暴露给模型使用。

3.6 Agent 和普通聊天机器人有什么区别？

普通聊天机器人主要是问答，用户问一句，它答一句。

Agent 更强调任务执行能力。它可以理解用户目标，拆解步骤，调用工具，读取知识库，保存状态，并根据执行结果继续下一步。

简单说：

聊天机器人偏回答，Agent 偏完成任务。

3.7 工作流是什么？

工作流就是把一个复杂任务拆成多个节点，然后按顺序或条件执行。

比如一个 AI 简历助手可以设计成：

1. 接收用户简历。
2. 判断岗位类型。
3. 检索岗位知识库。
4. 生成优化建议。
5. 输出最终简历。

扣子、Coze 这类平台的工作流，通常就是把模型节点、知识库节点、插件节点、条件判断节点连接起来，实现业务自动化。

3.8 SSE 流式响应是什么？为什么要用？

SSE 是 Server-Sent Events，服务端可以持续向客户端推送数据。

普通接口是等模型完整回答完再返回，用户等待时间会比较明显。SSE 可以边生成边返回，用户能实时看到文字输出，体验更接近 ChatGPT。

我的项目里提供了 `/chat/stream` 接口，返回 `text/event-stream`，并且会先返回 `chatId`，再返回消息内容，最后返回 `done` 事件。

3.9 Spring Boot 的核心优点是什么？

Spring Boot 的核心优点是简化 Spring 项目的配置和启动。

主要体现在：

1. 自动配置，减少大量手动配置。
2. 内置 Tomcat，可以直接运行。
3. starter 依赖简化包管理。
4. 配合注解开发 REST API 很方便。
5. 生态完整，容易集成数据库、Swagger、AI SDK 等。

我在项目里使用 Spring Boot 搭建接口层、服务层和配置层，用 Controller 暴露 `/chat`、`/chat/stream`、健康检查等接口。

3.10 Controller、Service、Repository 分别做什么？

Controller 主要负责接收 HTTP 请求、参数校验、返回响应。

Service 主要负责业务逻辑，比如对话流程、知识库检索、记忆构建。

Repository 主要负责数据访问，比如数据库增删改查、向量记录同步等。

这样分层的好处是职责清晰，代码更容易维护和测试。

3.11 GET 和 POST 有什么区别？

GET 通常用于查询数据，参数一般放在 URL 上，适合无副作用的请求。

POST 通常用于提交数据，参数放在请求体里，适合创建、修改或复杂查询。

比如聊天接口一般用 POST，因为用户消息较长，而且会产生会话记录。

3.12 HTTP 常见状态码有哪些？

常见状态码：

- 200：请求成功。
- 400：客户端参数错误。
- 401：未登录或认证失败。
- 403：没有权限。
- 404：资源不存在。
- 500：服务端异常。

接口联调时，我会先看状态码，再看响应体和后端日志定位问题。

3.13 数据库索引有什么作用？

索引可以提升查询速度，类似书的目录。

比如经常按照 `user_id`、`chat_id`、`source_path` 查询，就可以给这些字段加索引。

但索引不是越多越好，因为插入、更新数据时也要维护索引，会增加写入成本。

3.14 什么是事务？

事务是保证一组数据库操作要么全部成功，要么全部失败。

事务有 ACID 四个特性：

- 原子性：全部成功或全部失败。
- 一致性：事务前后数据状态符合业务规则。
- 隔离性：多个事务之间互不干扰。
- 持久性：事务提交后数据永久保存。

比如创建订单时，同时要扣库存、生成订单、写支付记录，如果中间一步失败，就应该整体回滚。

3.15 Vue 你了解哪些基础？

我了解 Vue 的基础用法，比如组件化、响应式数据、`v-if`、`v-for`、`v-model`、事件绑定、父子组件传参，以及通过 Axios 调用后端接口。

如果做 AI 对话页面，前端主要需要维护输入框、消息列表、loading 状态、`chatId`，以及处理普通接口或 SSE 流式返回。

3.16 Git 常用命令有哪些？

常用命令有：

- `git clone`：拉取项目。
- `git status`：查看状态。
- `git add`：暂存文件。
- `git commit`：提交代码。
- `git pull`：拉取远程更新。
- `git push`：推送代码。
- `git branch`：查看分支。
- `git switch` 或 `git checkout`：切换分支。

如果团队协作，我会先拉取最新代码，再开发自己的功能分支，提交前解决冲突并跑测试。

3.17 接口调试你一般怎么做？

我一般用 Apifox、Postman 或 Swagger 调试接口。

流程是：

1. 确认接口路径、请求方式和参数格式。
 2. 构造请求体。
 3. 查看状态码和返回结果。
 4. 如果报错，看后端日志。
 5. 对异常参数、空参数、重复请求等场景做测试。
 6. 最后整理接口文档或测试用例。
-

3.18 你怎么测试一个 AI 智能体是否可用？

我会从几个方面测试：

1. 正常问题能否回答。
2. 知识库问题是否能命中正确资料。
3. 无关问题是否会胡编。
4. 多轮对话是否能保持上下文。
5. 工具调用是否成功。
6. 边界情况，比如空输入、超长输入、敏感词。
7. 流式输出是否稳定。
8. 多次测试结果是否基本一致。

AI 应用测试不只看接口通不通，还要看效果是否符合业务预期。

3.19 扣子 / Coze 这类平台你怎么理解？

扣子这类平台主要是低代码智能体平台，可以通过 Prompt、知识库、插件、工作流等方式快速搭建 AI 应用。

它适合快速验证业务场景，比如客服问答、表单收集、内容生成、流程自动化等。

如果让我做，我会先梳理业务流程，再设计智能体角色和 Prompt，然后接入知识库和插件，最后用测试用例验证回答准确性、流程完整性和异常处理。

4. MostarManus 项目讲解稿

我的项目叫 **MostarManus AI Agent**，它是一个基于 Spring Boot 和 Spring AI 的智能学习与任务协作助手。

项目目标是做一个后端 AI Agent 服务，支持用户进行多轮对话、知识问答、学习计划辅助和任务协作。它不是简单调用大模型接口，而是在大模型外面加了一层完整的工程化能力，包括会话记忆、长期记忆、知识库检索、安全拦截、流式输出和工具调用。

项目主要分为六块。

第一块是接口层。我通过 `ChatController` 提供了 `/chat` 普通对话接口、`/chat/text` 文本接口和 `/chat/stream` 流式接口。流式接口用 SSE 实现，可以让前端边接收边展示模型回答。

第二块是 Agent 主流程。核心类是 `MostarManus`，它负责构建 Prompt、调用大模型、拼接上下文、保存对话记录，并根据不同记忆模式选择普通记忆或 Palace 记忆模式。

第三块是本地会话记忆。我实现了 `FileBasedChatMemory`，用 Kryo 把用户和助手的历史消息序列化到本地文件中。每次对话时，会取最近几轮对话拼进 Prompt，保证多轮对话有连续性。同时我对 `chatId` 做了白名单校验，只允许字母、数字、下划线和短横线，避免路径穿越问题。

第四块是 RAG 知识库。项目支持读取本地 Markdown 文档，把文档切分成 chunk，再通过 embedding 存到 PostgreSQL + pgvector。用户提问时，系统会从向量库里检索相关知识片段，并拼接到 Prompt 中，让模型基于本地资料回答。

第五块是长期记忆和记忆宫殿。项目里设计了长期记忆抽取、检索和 Palace 记忆模式，可以把用户偏好、历史任务等信息沉淀下来，在后续对话中召回相关记忆，让回答更个性化。

第六块是安全和稳定性。我做了敏感词短路拦截、异常提示封装、健康检查接口，并写了部分单元测试，比如计算器工具测试、时间工具测试、聊天记忆测试、知识切分测试、Palace 检索测试等。

这个项目让我比较熟悉 AI Agent 后端的基本链路：

用户请求进入 Controller，经过 Agent 构建 Prompt，检索知识库和记忆，调用大模型，再保存上下文并返回结果。

5. 项目架构简述

项目技术栈：

- Java 21
- Spring Boot 3
- Spring AI
- OpenAI 兼容模型接口
- PostgreSQL
- pgvector
- Kryo
- Maven
- Knife4j / Swagger

核心模块：

- `ChatController`：聊天接口，包含普通对话和 SSE 流式对话。
- `MostarManus`：Agent 主流程，负责 Prompt 构建、模型调用、上下文拼接和记忆写入。
- `FileBasedChatMemory`：本地会话记忆，用 Kryo 序列化保存聊天记录。
- `KnowledgeIngestionService`：本地知识库导入、切分、入向量库。
- `KnowledgeSearchService`：根据用户问题进行向量相似度检索。
- `MemoryContextBuilder`：构建长期记忆上下文。
- `PalaceContextBuilder`：构建 Palace 记忆上下文。
- `CalculatorTools`、`MemoryTools` 等：通过工具函数扩展 Agent 能力。
- `ProhibitedWordsAdvisor`：敏感词拦截。

6. 项目追问准备

6.1 你的项目最大亮点是什么？

我觉得最大亮点是把 AI 对话做成了一个相对完整的工程化 Agent，而不是简单调 API。

项目里有普通对话、SSE 流式输出、RAG 知识库、短期会话记忆、长期记忆、安全校验和测试用例。这些能力组合起来，比较接近真实业务里的 AI 助手后端。

6.2 你的项目里的 RAG 怎么实现？

我先读取本地 Markdown 文档，然后用 `KnowledgeChunker` 按长度和重叠字符数切分文本，再把每个片段包装成 Spring AI 的 `Document`，附带标题、路径、分类、`chunkIndex` 等 metadata，最后写入 `pgvector`。

查询时，通过 `KnowledgeSearchService` 调用 `VectorStore.similaritySearch`，按相似度找出 topK 文档片段，再拼进 Prompt 里给模型参考。

6.3 为什么要做文本切分？

因为文档通常比较长，不能直接全部放进 Prompt，也不利于向量检索。

切分后，每个 chunk 都是一个相对独立的语义片段，检索时能更精确地命中相关内容。

同时我设置了 overlap，也就是相邻片段之间保留一部分重叠内容，避免上下文被切断。

6.4 你为什么需要做会话记忆？

因为大模型接口本身通常是无状态的，如果每次只传当前问题，模型不知道前面聊了什么。

所以我做了本地会话记忆，把用户和助手的历史消息保存下来。下次请求时取最近几轮拼进 Prompt，这样模型就能理解上下文。

6.5 你怎么保证 chatId 安全？

因为会话记录是落到本地文件的，如果 chatId 不校验，可能出现路径穿越风险，比如传入 `../../xxx`。

所以我在 `FileBasedChatMemory` 里用正则限制 chatId，只允许字母、数字、下划线和短横线，不合法就直接抛异常。

6.6 为什么使用 SSE？

AI 回答通常比较慢，如果等完整结果返回，用户体验会比较差。

SSE 可以边生成边返回，让前端实时展示内容。它比 WebSocket 更轻量，适合服务端单向推送模型输出的场景。

6.7 你项目中遇到过什么问题？

一个典型问题是 AI 回答容易脱离本地知识，或者 RAG 没命中时模型会泛化回答。

我的处理方式是：

1. Prompt 中明确要求只在相关时使用本地知识。
 2. 检索时设置 topK 和相似度阈值。
 3. 文档切分时保留 overlap，减少上下文断裂。
 4. 对未命中场景单独设计测试用例，观察模型是否胡编。
-

6.8 如果让你接入扣子平台，你会怎么做？

我会先看业务场景，再拆流程。

比如要做一个企业知识库问答机器人，我会这样做：

1. 整理业务文档，上传知识库。
 2. 设计智能体角色和回答边界。
 3. 配置 Prompt，要求回答基于知识库。
 4. 如果有外部系统，就用插件或 HTTP 节点调用接口。
 5. 设计测试问题集，覆盖正常问题、边界问题和未命中问题。
 6. 根据测试结果调整知识库、Prompt 和流程节点。
-

7. AI 岗位八股速记

- **RAG**：先检索知识，再让模型回答，减少幻觉。
 - **Embedding**：把文本转成向量，用来计算语义相似度。
 - **向量数据库**：存储文本向量，支持相似度搜索，比如 pgvector。
 - **Prompt**：给模型的指令，包括角色、任务、约束和上下文。
 - **Function Calling**：让模型按需调用外部函数或工具。
 - **Agent**：能理解任务、规划步骤、调用工具并返回结果的 AI 应用。
 - **Workflow**：把多个节点按流程串起来，比如输入、判断、知识检索、模型生成、接口调用。
 - **SSE**：服务端持续推送数据，适合 AI 流式输出。
 - **接口联调**：确认路径、方法、参数、响应、异常和日志。
 - **AI 测试重点**：准确性、稳定性、边界输入、多轮上下文、知识命中、工具调用、异常处理。
-

8. 前后端基础速记

8.1 HTML、CSS、JavaScript

HTML 负责页面结构，CSS 负责页面样式，JavaScript 负责页面交互。

比如一个聊天页面，HTML 定义输入框和消息列表，CSS 控制布局和颜色，JavaScript 负责发送请求、接收响应、更新页面内容。

8.2 Vue

Vue 是前端框架，核心是组件化和响应式。

常用知识点：

- `v-if`: 条件渲染。
 - `v-for`: 列表渲染。
 - `v-model`: 双向绑定。
 - `@click`: 事件绑定。
 - `props`: 父组件传值给子组件。
 - `axios`: 调用后端接口。
-

8.3 RESTful API

RESTful API 是一种接口设计风格。

常见写法:

- `GET /users`: 查询用户列表。
 - `GET /users/{id}`: 查询单个用户。
 - `POST /users`: 创建用户。
 - `PUT /users/{id}`: 更新用户。
 - `DELETE /users/{id}`: 删除用户。
-

8.4 JSON

JSON 是前后端常用的数据交换格式。

示例:

```
{
  "message": "你好",
  "chatId": "test001",
  "memoryMode": "default"
}
```

9. 行为面试问题准备

9.1 你学习能力怎么样?

我的学习方式是先跑通最小 Demo, 再看原理和源码, 最后用项目做一遍。

比如学习 Spring AI 和 RAG 时, 我不是只看文档, 而是自己做了 MostarManus 项目, 把模型调用、知识库检索、会话记忆、工具调用和接口测试都实现了一遍。这样遇到实际业务时, 我能更快知道从哪里入手。

9.2 如果你不会某个工具怎么办?

我会先看官方文档和示例, 快速跑通一个最小可用版本, 然后再结合业务需求扩展。

如果是扣子、Coze 这类平台, 我会先搞清楚智能体、知识库、插件、工作流几个核心概念, 然后用测试用例验证每个节点是否符合预期。

9.3 你怎么和团队协作？

我会先明确需求和接口约定，比如字段、返回格式、错误码。开发过程中及时同步进度和问题，提交代码前自己先测试一遍。

如果遇到接口联调问题，我会先判断是前端参数、后端逻辑、网络请求还是数据问题，再把复现步骤和日志整理出来沟通。

10. 可以反问面试官的问题

1. 公司目前主要用扣子做哪类业务场景，是客服、销售、办公自动化还是内部知识库？
 2. 这个岗位更偏智能体搭建、工作流配置，还是也会参与 Spring Boot 接口开发？
 3. AI 应用上线前，公司一般怎么做效果评测和验收？
 4. 目前知识库是文档型、数据库型，还是需要对接第三方系统？
 5. 团队对新人更看重学习能力、测试落地能力，还是代码开发能力？
-

11. 面试前 30 分钟速背版

1. 这个岗位是 AI 应用落地岗，不是纯算法岗。
 2. 我会 Spring Boot 接口、数据库基础、接口联调，也了解前端基础。
 3. 我做过 MostarManus AI Agent 项目。
 4. 项目包含对话接口、SSE 流式输出、RAG、pgvector、会话记忆、长期记忆、工具调用和安全拦截。
 5. RAG 的流程是：文档切分、向量化、入库、检索、拼 Prompt、模型回答。
 6. Function Calling 是让模型调用外部工具，从回答问题扩展到执行任务。
 7. SSE 用来实现 AI 回答的流式输出。
 8. AI 应用测试要看准确性、稳定性、多轮上下文、知识命中、工具调用和异常处理。
 9. 扣子 / Coze 可以通过智能体、知识库、插件和工作流快速搭建业务方案。
 10. 我的优势是学习能力强，做过 AI Agent 项目，能从接口、流程和测试角度推动 AI 应用落地。
-

12. 最推荐你背的一段话

我理解这个岗位更偏 AI 应用落地，不是单纯做算法模型。我自己做过一个基于 Spring Boot 和 Spring AI 的 MostarManus 智能体项目，里面实现了普通对话、SSE 流式输出、RAG 知识库检索、pgvector 向量存储、会话记忆、长期记忆和工具调用。

通过这个项目，我熟悉了从用户请求到 Agent 处理，再到知识检索、上下文拼接、大模型调用和结果返回的完整链路。如果入职后需要使用扣子或类似平台搭建智能体和自动化工作流，我可以先快速理解业务流程，再配置知识库、Prompt、插件和测试用例，重点保证功能能跑通、效果可验证、流程稳定。